HIGH PERFORMANCE COMPUTING-ASS_12

NAME: J.Varshini

HALL TICKET NO: 2303A51758

BATCH NO:11

Task 1: CUDA Host-Device Data Transfer & Vector Addition
Objective: Learn how to allocate memory on the GPU (Island) and transfer data from the CPU (Mainland) using Google Colab.
Step 1: Open a new Google Colab Notebook. Go to Runtime -> Change runtime type and select T4 GPU.
Step 2: Install the CUDA plugin by running this in the first cell:
!pip install nvcc4jupyter
%load_ext nvcc4jupyter

Step 3: Create a new cell, type %%cuda at the top, and paste the following Vector Addition code:
%%cuda
#include <iostream>

_global_void addVectors(int n, float *d_A, float *d_B, float *d_C) { int workerID = blockIdx.x * blockDim.x +

01-04-2026 EOD

```
threadIdx.x; if
(workerID < n) {
d_C[workerID] =
d_A[workerID] +
d_B[workerID];
}
}

int main(void) { int N = 1000000; int
size = N * sizeof(float); float *h_A =
new float[N], *h_B = new float[N],
*h_C = new float[N];

for (int i = 0; i < N; i++) { h_A[i] = 1.0f; h_B[i]
= 2.0f; }

float *d_A, *d_B, *d_C; cudaMalloc(&d_A,
size); cudaMalloc(&d_B, size);
cudaMalloc(&d_C, size);

cudaMemcpy(d_A, h_A, size,
cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size,
cudaMemcpyHostToDevice);

int blockSize = 256;

int numBlocks = (N + blockSize - 1) / blockSize;
addVectors<<<numBlocks, blockSize>>>(N, d_A,
d_B, d_C);

cudaMemcpy(h_C, d_C, size,
cudaMemcpyDeviceToHost);
std::cout << "Data shipped back! First
Answer: " << h_C[0] << std::endl;
```

cudaFree(d_A); cudaFree(d_B); cudaFree(d_C); delete[]
h_A; delete[] h_B; delete[] h_C;
return 0;
}

```
%%cuda
#include <iostream>

__global__ void addVectors(int n, float *d_A, float *d_B, float *d_C) {

    int workerID = blockIdx.x * blockDim.x + threadIdx.x;

    if (workerID < n) {
        d_C[workerID] = d_A[workerID] + d_B[workerID];
    }
}

int main(void) {

    int N = 1000000;
    int size = N * sizeof(float);

    float *h_A = new float[N];
    float *h_B = new float[N];
    float *h_C = new float[N];

    // Initialize arrays
    for (int i = 0; i < N; i++) {
        h_A[i] = 1.0f;
        h_B[i] = 2.0f;
    }

    float *d_A, *d_B, *d_C;

    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);

    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    int blockSize = 256;
    int numBlocks = (N + blockSize - 1) / blockSize;

    addVectors<<<numBlocks, blockSize>>>(N, d_A, d_B, d_C);

    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    std::cout << "Data shipped back! First Answer: " << h_C[0] << std::endl;

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    delete[] h_A;
    delete[] h_B;
    delete[] h_C;

    return 0;
}
```

Step 4: Run the cell and take a screenshot of the output.

```
•••  Data shipped back! First Answer: 3
```

Task 2: OpenMP Offload (Asynchronous Execution)
Objective: Run an OpenMP GPU offload program in WSL Ubuntu using the nowait clause to see how the CPU can multitask.
Step 1: Open your WSL Ubuntu terminal.
Step 2: Create a file named nowait.cpp using nano nowait.cpp and paste the following code:

```
#include <iostream>
#include <omp.h>
#include <unistd.h>

int main() {
```

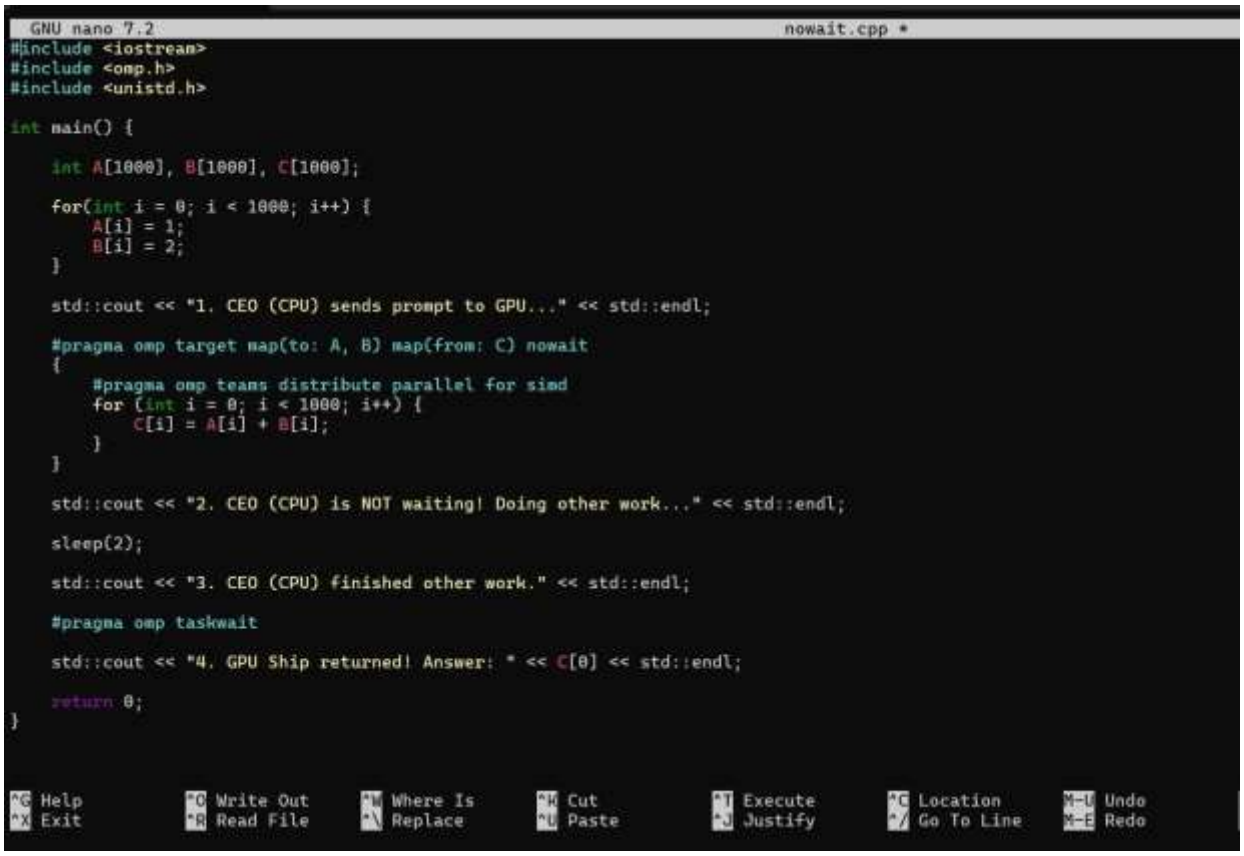
01-04-2026 EOD

```
int A[1000], B[1000], C[1000]; for(int
i = 0; i < 1000; i++) { A[i] = 1;
B[i] = 2; }

std::cout << "1. CEO (CPU) sends prompt to
GPU..." << std::endl;

#pragma omp target map(to: A, B) map(from:
C) nowait
{
#pragma omp teams distribute parallel for
simd
for (int i = 0; i < 1000; i++) {
C[i] = A[i] + B[i];
}
}
```

std::cout << "2. CEO (CPU) is NOT waiting!
Doing other work..." << std::endl; sleep(2);
std::cout << "3. CEO (CPU) finished other
work." << std::endl;

#pragma omp taskwait std::cout << "4. GPU
Ship returned! Answer:
" << C[0] << std::endl;

return 0;
}



Step 3: Compile using
clang++ -fopenmp nowait.cpp -o my_program

```
nithish@LAPTOP-A2FOJ865:~$ nano nowait.cpp
nithish@LAPTOP-A2FOJ865:~$ clang++ -fopenmp nowait.cpp -o my_program
Command 'clang++' not found, but can be installed with:
sudo apt install clang
```

```
nithish@LAPTOP-A2FOJ865:~$ clang++ -fopenmp nowait.cpp -o my_program
```

Step 4: Run ./my_program and take a screenshot of the terminal.

```
nithish@LAPTOP-A2FOJ865:~$ clang++ -fopenmp nowait.cpp -o my_program
nithish@LAPTOP-A2FOJ865:~$ ./my_program
1. CEO (CPU) sends prompt to GPU...
2. CEO (CPU) is NOT waiting! Doing other work...
3. CEO (CPU) finished other work.
4. GPU Ship returned! Answer: 3
nithish@LAPTOP-A2FOJ865:~$
```

Task 3: Conceptual Questions

Answer the following based on your executions:

1. **In CUDA, what specific function is used to send data from the Mainland (CPU) to the Island (GPU)?**

In CUDA programming, the data transfer from the CPU (Host/Mainland) to the GPU (Device/Island) is performed using the cudaMemcpy() function. This function copies data between host memory and device memory. When transferring data from the CPU to the GPU, the flag cudaMemcpyHostToDevice is used. It ensures that the input data stored in the CPU memory becomes available for computation on the GPU.

2. **What happens to the GPU memory if a programmer forgets to write cudaFree() at the end of the program?**

If a programmer forgets to call cudaFree(), the memory allocated on the GPU will not be released properly after the program finishes execution. This results in a memory leak, where GPU memory remains occupied unnecessarily. Over time, repeated executions may consume all available GPU memory, which can slow down the system or prevent other GPU programs from running correctly. Therefore, freeing GPU memory using cudaFree() is an important step in CUDA programming.

3. **In the OpenMP nowait program, why does the CPU print "Doing other work..." before the GPU finishes its math?**

In the OpenMP program, the nowait clause allows the CPU to continue executing the next instructions without waiting for the GPU computation to complete. Once

the CPU sends the task to the GPU, the GPU performs the calculations independently in parallel. Meanwhile, the CPU proceeds with other operations, which is why the message "Doing other work..." appears before the GPU finishes its computation. This demonstrates parallel execution and efficient utilization of system resources.

Task 4: GPU Performance Analysis (Global vs Shared Memory)

Objective: Compare the execution time of slow Global memory vs ultra-fast Shared memory in Google Colab.

Step 1: In a new Google Colab cell (with %%cuda at the top), paste the Matrix Multiplication code that uses both matrixMulGlobal and matrixMulShared (The Architect Way).

```
%%cuda
#include <iostream>
#include <cuda_runtime.h>

#define N 1024
#define TILE_SIZE 16

// THE ROOKIE WAY: GLOBAL MEMORY
__global__void matrixMulGlobal(float *A, float
*B, float *C) { int row = blockIdx.y
* blockDim.y + threadIdx.y; int col
= blockIdx.x * blockDim.x +
threadIdx.x; if (row < N && col <
N) { float sum = 0.0f; for (int i = 0;
i < N; i++) { sum += A[row * N + i]
* B[i * N + col];
}
C[row * N + col] = sum;
}
}

// THE ARCHITECT WAY: SHARED MEMORY _global
void matrixMulShared(float *A, float
*B, float *C) {
```

```
_____shared_____float
shared_A[TILE_SIZE][TILE_SIZE]; __shared
float shared_B[TILE_SIZE][TILE_SIZE]; int bx =
blockIdx.x; int by = blockIdx.y; int tx = threadIdx.x;
int ty = threadIdx.y;

int row = by * TILE_SIZE + ty; int
col = bx * TILE_SIZE + tx; float
sum = 0.0f;

for (int i = 0; i < (N / TILE_SIZE); i++) {
shared_A[ty][tx] = A[row * N + (i *
TILE_SIZE + tx)]; shared_B[ty][tx] =
B[(i * TILE_SIZE + ty) * N + col];
__syncthreads();

for (int k = 0; k < TILE_SIZE; k++) {
sum += shared_A[ty][k] *
shared_B[k][tx];
}
__syncthreads();
}
C[row * N + col] = sum;
}

int main() { int size = N * N * sizeof(float);
float *h_A = new float[N * N]; float *h_B
=

new float[N * N];
for (int i = 0; i < N * N; i++) { h_A[i] =
1.0f; h_B[i] = 2.0f; }

float *d_A, *d_B, *d_C; cudaMalloc(&d_A,
size); cudaMalloc(&d_B, size);
cudaMalloc(&d_C, size);
cudaMemcpy(d_A, h_A, size,
```

```
cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size,
cudaMemcpyHostToDevice);

dim3 threads(TILE_SIZE, TILE_SIZE); dim3
blocks(N / TILE_SIZE, N / TILE_SIZE);
cudaEvent_t start, stop; cudaEventCreate(&start);
cudaEventCreate(&stop);

cudaEventRecord(start);
matrixMulGlobal<<<blocks, threads>>>(d_A,
d_B, d_C); cudaEventRecord(stop);
cudaEventSynchronize(stop); float ms_global =
0; cudaEventElapsedTime(&ms_global, start,
stop);

cudaEventRecord(start);
matrixMulShared<<<blocks, threads>>>(d_A,

d_B, d_C); cudaEventRecord(stop);
cudaEventSynchronize(stop); float ms_shared =
0; cudaEventElapsedTime(&ms_shared, start,
stop);

std::cout << "Time using Slow Global Memory:
" << ms_global << " ms\n"; std::cout << "Time
using Fast Shared Memory:
" << ms_shared << " ms\n"; std::cout <<
"SPEEDUP FACTOR: " <<
(ms_global / ms_shared) << "x Faster!\n";

cudaFree(d_A); cudaFree(d_B); cudaFree(d_C); delete[]
h_A; delete[] h_B;
return 0;
}
```

```
```

```
int main() {

    int size = N * N * sizeof(float);

    float *h_A = new float[N * N];
    float *h_B = new float[N * N];

    for (int i = 0; i < N * N; i++) {
        h_A[i] = 1.0f;
        h_B[i] = 2.0f;
    }

    float *d_A, *d_B, *d_C;

    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);

    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    dim3 threads(TILE_SIZE, TILE_SIZE);
    dim3 blocks(N / TILE_SIZE, N / TILE_SIZE);

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    // GLOBAL MEMORY TIMING
    cudaEventRecord(start);
    matrixMulGlobal<<<blocks, threads>>>(d_A, d_B, d_C);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    float ms_global = 0;
    cudaEventElapsedTime(&ms_global, start, stop);

    // SHARED MEMORY TIMING
    cudaEventRecord(start);
    matrixMulShared<<<blocks, threads>>>(d_A, d_B, d_C);
    cudaEventRecord(stop);
    cudaEventSynchronize(stop);

    float ms_shared = 0;
    cudaEventElapsedTime(&ms_shared, start, stop);

    std::cout << "Time using Slow Global Memory: "
              << ms_global << " ms\n";

    std::cout << "Time using Fast Shared Memory: "
              << ms_shared << " ms\n";

    std::cout << "SPEEDUP FACTOR: "
              << (ms_global / ms_shared)
              << "x Faster!\n";

    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);

    delete[] h_A;
    delete[] h_B;

    return 0;
}
```

Step 2: Run the cell. The program uses cudaEventElapsedTime to act as a stopwatch.

```
Time using Slow Global Memory: 31.015 ms
Time using Fast Shared Memory: 5.14662 ms
SPEEDUP FACTOR: 6.02628x Faster!
```

Step 3: Observe the output at the bottom of the terminal.
Write down the following readings:
* Time using Slow Global Memory: 31.015 ms
* Time using Fast Shared Memory: 5.14662 ms
* SPEEDUP FACTOR:6.02628 x Faster

Step 4: Take a screenshot of these timing results.


```
Time using Slow Global Memory: 31.015 ms
Time using Fast Shared Memory: 5.14662 ms
SPEEDUP FACTOR: 6.02628x Faster!
```


Task 5: The IF/ELSE Trap (Warp Divergence)
Objective: Observe what happens when GPU threads inside the

same Warp try to take different if/else paths.
Step 1: In a new Colab cell, write %%cuda and paste this code:

```
%%cuda
#include <iostream>

_global_void divergenceTest(int *data) { int
workerID = threadIdx.x; // Worker 0 to
31 (One Warp)

// THE ROOKIE MISTAKE: Warp Divergence if
(workerID < 16) {
data[workerID] = data[workerID] * 2;
} else { data[workerID] =
data[workerID] + 5;
}
```

}
int main() { std::cout << "--- THE IF/ELSE TRAP ---" << std::endl;
std::cout << "Rule: All 32 workers in a Warp MUST share the same megaphone." << std::endl;
std::cout << "If they disagree, the GPU cuts its speed in half!" << std::endl; return 0;
}

```
%%cuda
#include <iostream>

__global__ void divergenceTest(int *data) {

    int workerID = threadIdx.x; // Workers 0 to 31 (one warp)
    if (workerID < 16) {
        data[workerID] = data[workerID] * 2;
    } else {
        data[workerID] = data[workerID] + 5;
    }
}

int main() {

    std::cout << "--- THE IF/ELSE TRAP ---" << std::endl;
    std::cout << "Rule: All 32 workers in a Warp MUST share the same megaphone." << std::endl;
    std::cout << "If they disagree, the GPU cuts its speed in half!" << std::endl;

    return 0;
}
```

Step 2: Run the cell and capture a screenshot.

```
--- THE IF/ELSE TRAP ---
Rule: All 32 workers in a Warp MUST share the same megaphone.
If they disagree, the GPU cuts its speed in half!
```

Task 6: Task 3: Observation Questions

Based on the performance metrics you just analyzed, answer the following:

1. Why is "Shared Memory" (The Architect Way) so much faster than "Global Memory" (The Rookie Way) in matrix Multiplication?

Shared Memory is faster because it is placed very close to the GPU cores, allowing threads to access data quickly. During matrix multiplication, the same data values are used multiple times by different threads. By storing these values in Shared

Memory, the GPU avoids repeatedly accessing Global Memory, which is slower and located farther away.

In contrast, Global Memory has higher access time, so frequent reads and writes reduce performance. Using Shared Memory improves data reuse and minimizes memory access delay, resulting in much faster execution.

2. What is the OpenMP/CUDA command used to make sure all workers finish loading data onto the whiteboard before they start doing math? (_syncthreads())

The command used is _syncthreads().

It works like a meeting point for all threads in a block. Every thread waits at this point until all other threads finish loading data into Shared Memory. Only after everyone reaches this stage do the threads continue with the computation. This ensures that calculations are performed using complete and correct data.

3. What is "Warp Divergence" and why does an if/else statement hurt GPU performance?

Warp Divergence happens when threads within the same warp take different execution paths because of conditions like if/else statements. Since GPU threads in a warp execute instructions together, the GPU cannot run both branches at the same time. Instead, it executes one branch while the other threads remain idle, and then switches to the second branch.

This reduces parallel efficiency and slows down performance. Therefore, minimizing conditional branching helps maintain better GPU speed.